PILA DE TECNOLOGÍA UTILIZADA

Frontend:

-Css: En este caso, hemos utilizado esta tecnología para darle estilo a nuestros archivos jsp y que quede acorde con la web que estamos desarrollando.

-Jsp: Esta tecnología ha sido usada para la realización del modelo vista controlador (mvc).

-JavaScript: Hemos seleccionado esta tecnología para crear scripts que en nuestro caso ha sido mostrar/ocultar contraseña

Backend:

-Java: Esta tecnología la hemos utilizado principalmente a la hora de desarrollar los servlet.

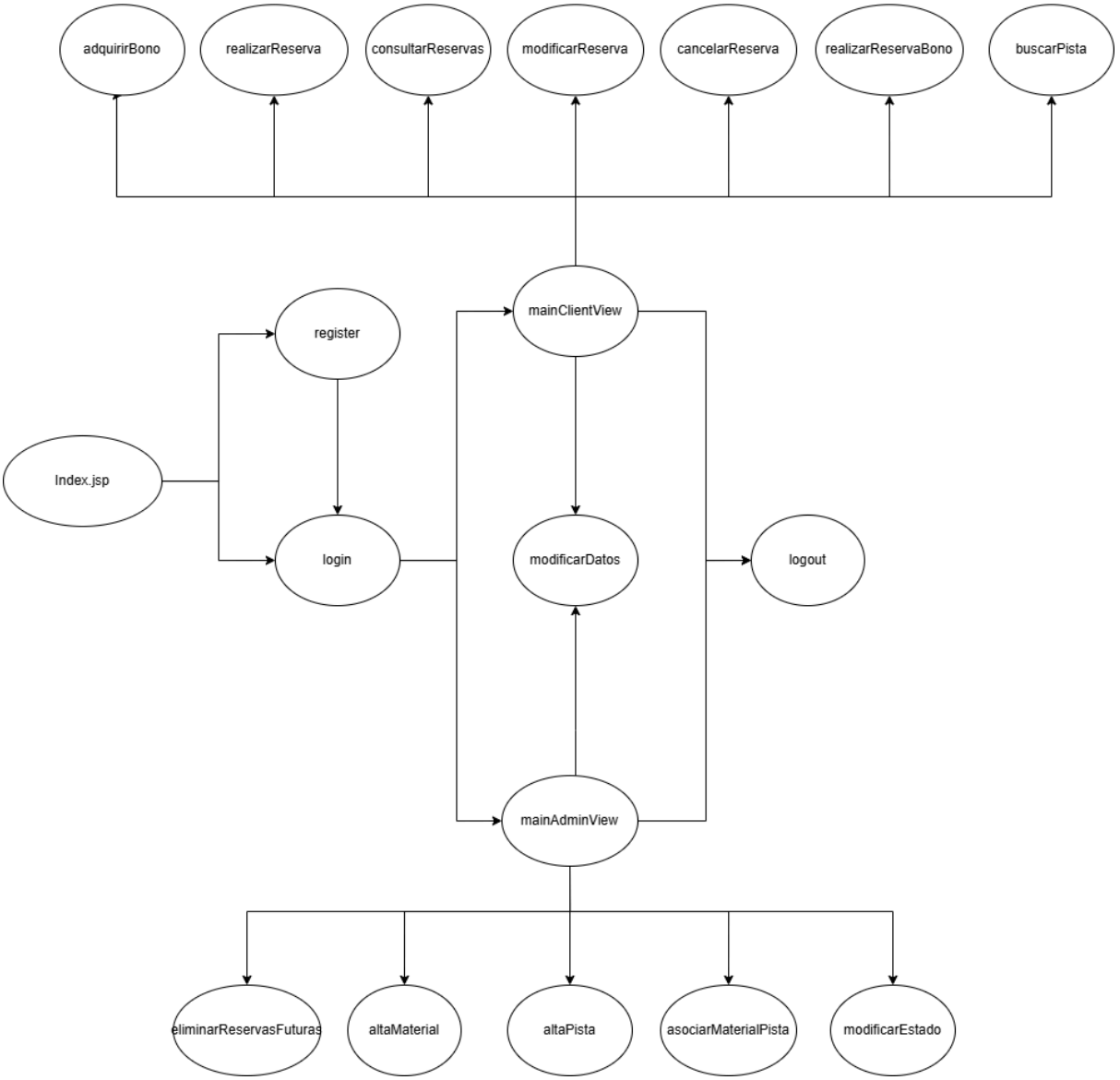
-JSP: (Lógica del lado del servidor): su salida es HTML, el código Java dentro del JSP se ejecuta en el servidor.

-Servlet: Tecnología de Java utilizada en el backend para manejar solicitudes y respuestas HTTP.

Infraestructura:

-El sistema está implementado y desplegado en un servidor Apache Tomcat 10.1.33, el cual se encarga de las peticiones de HTTP y ejecutar los componentes.

MAPA DE NAVEGACIÓN

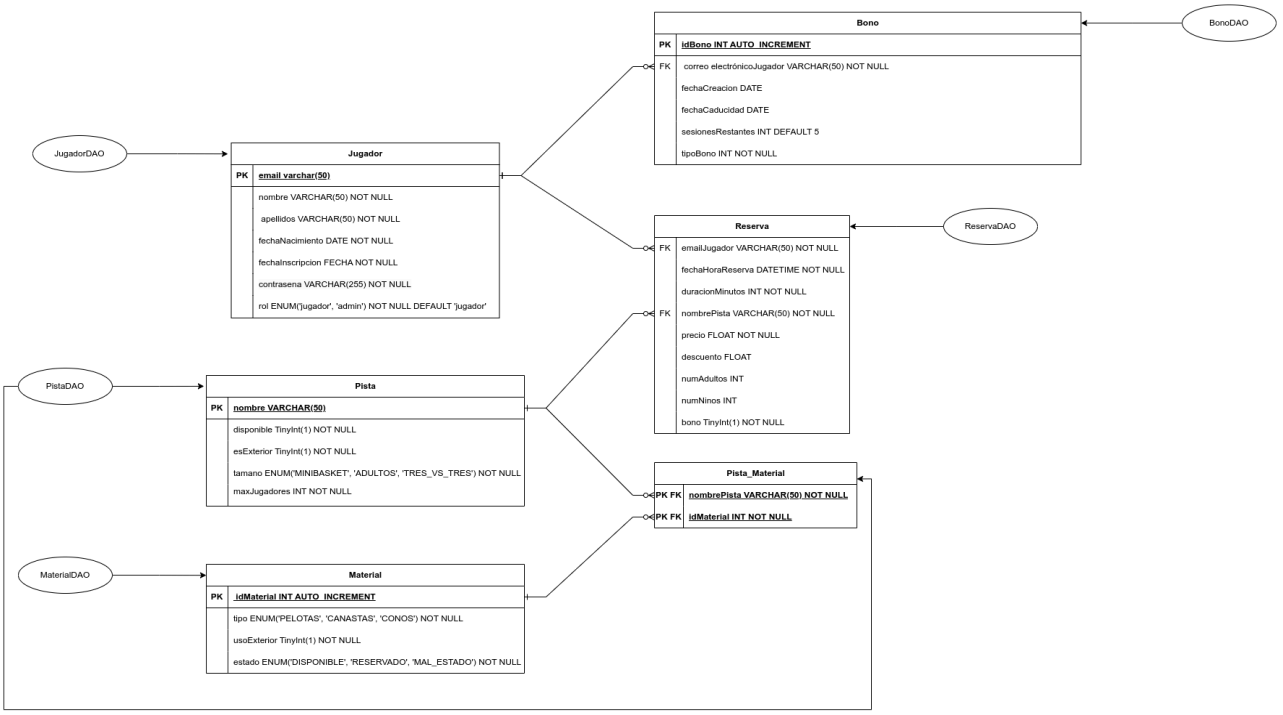


En la arquitectura de nuestra aplicación, existen diferentes formas de gestionar la interacción entre las vistas y los datos provenientes de la base de datos, dependiendo de la funcionalidad que se quiera implementar. Algunas vistas requieren que los datos necesarios sean cargados desde la base de datos antes de que estas puedan mostrarse en el navegador del usuario. Sin embargo, también existen situaciones en las que el usuario accede primero a una vista para introducir datos o interactuar con un formulario, y posteriormente el servlet procesa esa información

Por ejemplo en la funcionalidad de adquirir bono, el usuario accede directamente a una vista donde puede observar los campos que tiene que rellenar para poder adquirir un bono. Posteriormente, esta información se envía al servlet, que procesa la solicitud y consulta la base de datos en función de los criterios proporcionados.

Por otro lado, en la funcionalidad de realizar una reserva, el flujo funciona de manera distinta. Antes de que el usuario pueda acceder a la vista correspondiente para realizar la reserva, es necesario que se carguen previamente las pistas disponibles desde la base de datos. Este proceso es gestionado por el servlet, que obtiene los datos necesarios del modelo y los pasa a la vista para que el usuario pueda seleccionarlos.

ESQUEMA RELACIONAL DE LA BASE DE DATOS



INSTRUCCIONES DE ACCESO

Para acceder a la aplicación se necesita estar conectado a la base de datos mediante una vpn. Tras esto ya podremos hacer solicitudes y con ello poder iniciar sesión, registrarse y poder trabajar dentro de la propia aplicación. Para acceder por primera vez se necesita estar registrado. Una vez registrado ya podremos iniciar sesión y acceder a la aplicación y a sus funcionalidades. Para hacer pruebas de usuario hemos creado el siguiente usuario:

usuario:guillermito@gmail.com

contraseña:1234

Y para hacer pruebas de administrador hemos creado el siguiente usuario:

usuario:adminadmin@gmail.com

contraseña: admin

DECISIONES DE DISEÑO

Al principio tuvimos una serie de problemas, ya que no teníamos ningún campo para almacenar la contraseña del usuario, ni tampoco uno que diferenciara los roles de cliente y administrador. Por lo que nos vimos en la necesidad de añadir dos columnas nuevas en la tabla jugador de la base de datos.

Por otra parte, para gestionar estos 2 tipos de cuentas, unas para acceder como cliente y otras para acceder como administrador, en nuestro caso hemos decidido permitir seleccionar en la parte del registro el tipo de cuenta que quieres crear, así inicialmente se puedan crear ambos tipos de cuentas para probar todas las funcionalidades de nuestra página.

Otro cambio que introducimos fue añadir un nuevo campo en la tabla de reservas, es el campo bono. Su función es diferenciar cuando una reserva está realizada con bono y cuando no.

También hemos pensado que no sería lógico que un cliente pueda adquirir bonos de manera ilimitada, por lo que permitimos que un cliente solo pueda tener 1 bono máximo de cada tipo, de esta manera cuando consuma todas sus sesiones, este será eliminado de la base de datos y permitirá volver a obtener otro en caso de que lo requiera.

Entre alguno de los problemas que hemos encontrado a la hora del desarrollo ha sido la parte de conseguir redirigir correctamente de una vista a otra, ya que probamos distintas maneras hasta que encontramos la que mejor nos funcionaba, inicialmente probamos con rutas absolutas pero en función de donde accediesemos había rutas que no conseguía cogerlas correctamente, por lo que finalmente descubrimos la función request.getContextPath() la cual ha sido la que nos ha ayudado con el tema de la redirección de rutas.

Otro problema que encontramos fue a la hora de utilizar los servlets, ya que la librería javax ya no es compatible con los servidores de apache tomcat a partir de la versión 9. Por lo cual tuvimos que utilizar la librería de jakarta, que tiene el mismo funcionamiento que la librería javax. Esto nos retrasó el ritmo de trabajo puesto que inicialmente pensábamos que el error se debía a un uso incorrecto de los servlets. Una vez que detectemos el origen del problema logremos continuar con el desarrollo de la práctica sin mayores contratiempos.

Además, hemos decidido añadir en EliminarReservasFuturas otra opción que se encargue de borrar una reserva en específico, aparte de la opción que ya se pedía de borrar todas las reservas futuras.